Universidad del Istmo
FING
Víctor Vargas
Ingeniería Electrónica y Telecomunicaciones



Examen Final

ALU+ (HDL, síntesis y diseño físico)

Sofia del Pilar Martínez Chajón

Parte 1, HDL + ALU + Adders

Se diseñó en SystemVerilog una ALU+ basada en la ALU de Harris & Harris, a la que se le agregó un divisor, un bit más de control, compuertas AND/OR aisladas por un bus triestado y la circuitería de banderas NZCV. Todos los módulos usan el parámetro `WIDTH` (8 bits por defecto), de modo que la misma descripción se instancia a 16 o 32 bits con `alu_plus #(.WIDTH(16))` sin cambiar la arquitectura. A esta variante le corresponden el **Prefix Adder** y el **Divisor**.

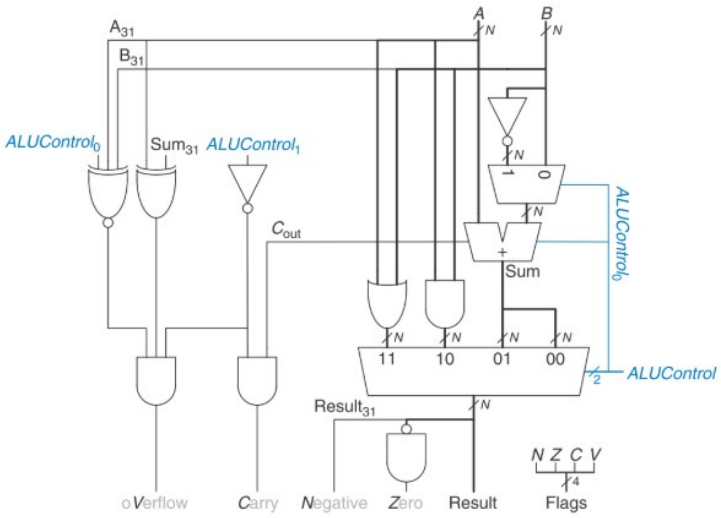


Figura 1. *ALU de referencia (Harris & Harris) sobre la que se construyó la ALU+.*

1.1 Tabla de operaciones

Agregar el divisor aumentó ALUControl de 2 a 3 bits. La bandera N es el bit más significativo del resultado y Z vale 1 cuando el resultado es cero, en todas las operaciones.

ALUControl	Operación	Resultado	C	V
000	Suma	A + B	Carry de salida	Overflow con signo
001	Resta	A − B	1 = sin borrow	Overflow con signo
010	AND (triestado)	A & B	0	0
011	OR (triestado)	A | B	0	0
100	Reservado (multiplicador, otro grupo)	0	0	0
101	Divisor: cociente	A / B	0	1 si B = 0
110	Divisor: residuo	A % B	0	1 si B = 0
111	Todos los bits en 0	0	0	0

1.2 Estructura básica

A y B entran en paralelo al sumador, a la unidad AND/OR y al divisor. B pasa por un multiplexor 2:1 que elige B o ~B según ALUControl[0]; ese mismo bit entra como carry de entrada, así que A − B = A + ~B + 1 (complemento a 2) y un solo sumador hace suma y resta. Un multiplexor 8:1 controlado por ALUControl elige qué unidad entrega el resultado.

1.3 Multiplexores con operador ternario

Los multiplexores se describen con `assign y = s ? d1 : d0;`. Esta forma describe exactamente un multiplexor, por lo que la herramienta de síntesis lo mapea directamente a celdas MUX de la librería. El multiplexor 8:1 es un árbol de tres niveles de ternarios indexados por ALUControl.

1.4 Sumador Prefix (Sklansky)

Cada bit calcula si genera carry (g = a·b) o si lo propaga (p = a ⊕ b); el carry de entrada se trata como la columna −1. La celda prefijo combina rangos vecinos de columnas:

$$G(i:j) = G(i:k) + P(i:k) \cdot G(k-1:j) \qquad P(i:j) = P(i:k) \cdot P(k-1:j)$$

En el nivel l se combinan bloques de 2^l columnas. Tras $\log_2(N+1)$ niveles cada bit conoce su carry y la suma es S(i) = p(i) ⊕ C(i). El retardo crece con $\log_2 N$ en lugar de N, como en el ripple carry: 4 niveles a 8 bits y 6 niveles a 32 bits. Por ejemplo, en la suma 63 + 1 el carry generado en el bit 0 alcanza 1, luego 3 y luego 6 columnas en niveles sucesivos del árbol.

1.5 Divisor de arreglo (restoring)

El divisor implementa la división larga binaria con una fila por bit del cociente, del más significativo al menos significativo:

1. Se baja el siguiente bit de A al residuo parcial: T = {R, A[i]}.
2. Se resta B con un Prefix Adder de N+1 bits: D = T + ~B + 1.
3. El carry de salida es el bit del cociente: vale 1 si T ≥ B.
4. Un multiplexor 2:1 conserva D si la resta fue válida o restaura T si fue negativa.

Para 100 / 7 el residuo parcial pasa por 0, 0, 1, 3, 6, 5, 4, 1 y 2, y se obtiene cociente 14 y residuo 2. Si B = 0, el cociente sale en unos, el residuo es A y se levanta la bandera V. Como son N restas en serie, el divisor es el camino más largo de la ALU.

1.6 ALUControl de 3 bits

El cociente sale en 101, el residuo en 110 y la salida 111 entrega todos los bits en 0, útil como operación nula o para limpiar un registro. La salida 100 queda reservada para el multiplicador, que corresponde a otro grupo, y entrega 0.

1.7 AND/OR con bus triestado

En la ALU original las compuertas AND y OR reciben A y B siempre y conmutan aunque la operación sea otra. En esta ALU+ la unidad lógica solo propaga la operación cuando se invoca:

- Buffers triestado a la entrada: A y B solo llegan a la AND cuando ALUControl = 010 y a la OR cuando es 011; si no, sus entradas quedan en alta impedancia (z).
- Un buffer triestado a la salida entrega el resultado de AND u OR al bus `logic_bus` y lo deja en z en cualquier otra operación. Hay un solo driver por bit, por lo que no existe contención en el bus.

En silicio, un nodo triestado se acompaña de un bus keeper para que no quede flotando a un voltaje intermedio.

1.8 Banderas NZCV

- **N:** copia el bit más significativo del resultado.
- **Z:** NOR de todos los bits del resultado.
- **C:** carry de salida del sumador, solo en suma y resta.
- **V:** en suma y resta indica overflow con signo, cuando los operandos (B ya invertido en la resta) tienen el mismo signo y el resultado sale con signo distinto; en el divisor indica división entre cero.

$$V = \neg(ALUControl[0] \oplus A[N-1] \oplus B[N-1]) \cdot (A[N-1] \oplus Sum[N-1])$$

1.9 Verificación

La ALU+ pasó todas las pruebas sin errores en 8, 16 y 32 bits, simuladas con VCS en el servidor de Synopsys. Un testbench autoverificable compara el resultado y las banderas NZCV de cada vector contra un modelo de referencia escrito con los operadores de SystemVerilog.

Configuración	Vectores	Errores
8 bits	524,584 (exhaustivo: 65,536 pares A, B × 8 operaciones)	0
16 bits	16,296 (casos de esquina + aleatorios)	0
32 bits	16,296 (casos de esquina + aleatorios)	0

Los casos de esquina incluyen 0, −1, 1, el máximo positivo, el mínimo negativo y la división entre cero. El demo `tb_demo.sv` muestra cada inciso con valores concretos (8 bits):

Prueba	Entrada	Resultado	NZCV
Suma	100 + 7	107	0000
Resta	100 − 7	93	0010
AND	100 & 7	4	0000
OR	100 | 7	103	0000
Cociente	100 / 7	14	0000
Residuo	100 % 7	2	0000
Todos en 0	ALUControl = 111	0	0100
División entre 0	100 / 0	255	1001
Overflow positivo	127 + 1	128	1001
Overflow negativo	−128 − 1	127	0011
Resta con borrow	3 − 5	254 (−2)	1000

En el bus triestado, con A = 0xF0 y B = 0x3C, `logic_bus` vale 00110000 en AND, 11111100 en OR y zzzzzzzz en las otras seis operaciones.

```
./run_all.sh vcs 2000
=== ALU+  WIDTH=8  (Prefix Adder + Divisor) ===
Verificaciones: 524584   Errores: 0
RESULTADO: PASS
=== ALU+  WIDTH=16  (Prefix Adder + Divisor) ===
Verificaciones: 16296   Errores: 0
RESULTADO: PASS
=== ALU+  WIDTH=32  (Prefix Adder + Divisor) ===
Verificaciones: 16296   Errores: 0
RESULTADO: PASS
[Pilix@synopsys-unis sim]$
```

Figura 2. *Regresión en VCS: PASS con 0 errores a 8, 16 y 32 bits.*

```
Pilix@synopsys-unis:~/alu_pl
[Pilix@synopsys-unis sim]$ vcs -sverilog -full64 -timescale=1ns/1ps ../rtl/*.sv
./demo.simv | sed -n '/\[1,5\]/,$p'
[1,5] Las 8 operaciones con A=100, B=7
  000 ADD  A=100 (01100100)  B=  7 (00000111) -> R=107 (01101011) NZCV=0000
  001 SUB  A=100 (01100100)  B=  7 (00000111) -> R= 93 (01011101) NZCV=0010
  010 AND  A=100 (01100100)  B=  7 (00000111) -> R=  4 (00000100) NZCV=0000
  011 OR   A=100 (01100100)  B=  7 (00000111) -> R=103 (01100111) NZCV=0000
  100 ---  A=100 (01100100)  B=  7 (00000111) -> R=  0 (00000000) NZCV=0100
  101 DIV  A=100 (01100100)  B=  7 (00000111) -> R= 14 (00001110) NZCV=0000
  110 REM  A=100 (01100100)  B=  7 (00000111) -> R=  2 (00000010) NZCV=0000
  111 ZER  A=100 (01100100)  B=  7 (00000111) -> R=  0 (00000000) NZCV=0100

[3] Prefix Adder: 63 + 1 (el carry del bit 0 se propaga nivel por nivel)
  R=64 NZCV=0000
  nivel 0 G = 000000010
  nivel 1 G = 000000010
  nivel 2 G = 000001110
  nivel 3 G = 001111110
  nivel 4 G = 001111110  (= carries C8..C0)

[4] Divisor 100 / 7 (residuo parcial por fila)
  r[0] =   0
  r[1] =   0
  r[2] =   1
  r[3] =   3
  r[4] =   6
  r[5] =   5
  r[6] =   4
  r[7] =   1
  r[8] =   2
  cociente=14 residuo=2
  101 DIV  A=100 (01100100)  B=  7 (00000111) -> R= 14 (00001110) NZCV=0000
  110 REM  A=100 (01100100)  B=  7 (00000111) -> R=  2 (00000010) NZCV=0000
  101 DIV  A=100 (01100100)  B=  0 (00000000) -> R=255 (11111111) NZCV=1001
  110 REM  A=100 (01100100)  B=  0 (00000000) -> R=100 (01100100) NZCV=0001

[6] Bus triestado de AND/OR (A=0xF0, B=0x3C)
  op=000 ADD  a_and=zzzzzzzz  a_or=zzzzzzzz  logic_bus=zzzzzzzz
  op=001 SUB  a_and=zzzzzzzz  a_or=zzzzzzzz  logic_bus=zzzzzzzz
  op=010 AND  a_and=11110000  a_or=zzzzzzzz  logic_bus=00110000
  op=011 OR   a_and=zzzzzzzz  a_or=11110000  logic_bus=11111100
```

Figura 3. *Demo por inciso en VCS: operaciones, Prefix Adder, divisor y bus triestado.*

```
[6] Bus triestado de AND/OR (A=0xF0, B=0x3C)
  op=000 ADD  a_and=zzzzzzzz  a_or=zzzzzzzz  logic_bus=zzzzzzzz
  op=001 SUB  a_and=zzzzzzzz  a_or=zzzzzzzz  logic_bus=zzzzzzzz
  op=010 AND  a_and=11110000  a_or=zzzzzzzz  logic_bus=00110000
  op=011 OR   a_and=zzzzzzzz  a_or=11110000  logic_bus=11111100
  op=100 ---  a_and=zzzzzzzz  a_or=zzzzzzzz  logic_bus=zzzzzzzz
  op=101 DIV  a_and=zzzzzzzz  a_or=zzzzzzzz  logic_bus=zzzzzzzz
  op=110 REM  a_and=zzzzzzzz  a_or=zzzzzzzz  logic_bus=zzzzzzzz
  op=111 ZER  a_and=zzzzzzzz  a_or=zzzzzzzz  logic_bus=zzzzzzzz

[7] Banderas NZCV
  000 ADD  A=127 (01111111)  B=  1 (00000001) -> R=128 (10000000) NZCV=1001
  000 ADD  A=128 (10000000)  B=128 (10000000) -> R=  0 (00000000) NZCV=0111
  001 SUB  A=  5 (00000101)  B=  5 (00000101) -> R=  0 (00000000) NZCV=0110
  001 SUB  A=  3 (00000011)  B=  5 (00000101) -> R=254 (11111110) NZCV=1000
  001 SUB  A=128 (10000000)  B=  1 (00000001) -> R=127 (01111111) NZCV=0011
  111 ZER  A=  0 (00000000)  B=  0 (00000000) -> R=  0 (00000000) NZCV=0100
```

Figura 4. *Demo por inciso (continuación): bus triestado y banderas NZCV.*

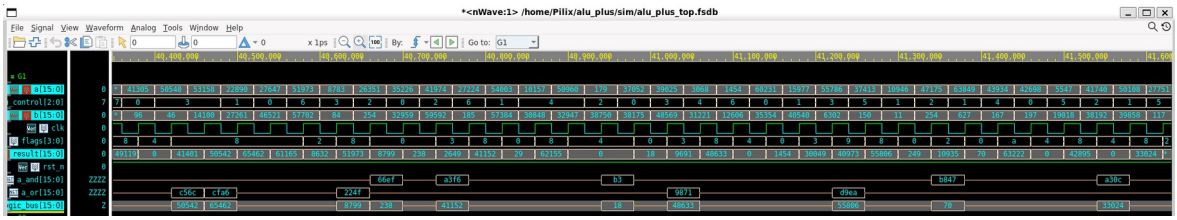


Figura 5. *Ondas en Verdi (16 bits): logic_bus, a_and y a_or toman valor solo con ALUControl = 2 (AND) o 3 (OR) y quedan en alta impedancia (z) en las demás operaciones.*

Parte 2, Physical Design + Synopsys

La ALU+ de 16 bits se implementó hasta GDSII en el servidor de Synopsys de la universidad, con el PDK de GF 150 nm y la librería de celdas estándar M31 LLSC733 (3.3 V, 7 tracks). El flujo completó floorplan, alimentación, placement, árbol de reloj y routing, con 0 errores de DRC, 0 errores de LVS y legalidad aprobada.

Herramienta	Versión	Uso
VCS + Verdi	X-2025.06 / Y-2026.03	Simulación RTL y ondas (FSDB)
Design Compiler	X-2025.06	Síntesis lógica
IC Compiler II	Y-2026.03	Diseño físico
IC Compiler II Library Manager	Y-2026.03	Conversión de la librería M31 a NDM

2.1 Preparación: sistema síncrono

`alu_plus_top.sv` coloca la ALU+ de 16 bits entre un banco de registros de entrada (A, B, ALUControl) y uno de salida (Result, NZCV), con reset asíncrono. Así toda ruta interna va de registro a registro y puede restringirse con un reloj; el resultado aparece dos ciclos después de aplicar la entrada. En VCS, el testbench con reloj aplicó 2,045 vectores con 0 errores y generó el archivo FSDB para Verdi.

```
[Pilix@synopsys-unis sim]$ cd ~/alu_plus
make sim | tail -8
Verificaciones: 2045   Errores: 0
RESULTADO: PASS
$finish called from file "../tb/tb_alu_plus_top.sv", line 117.
$finish at simulation time             81980000
           V C S   S i m u l a t i o n   R e p o r t
Time: 81980000 ps
CPU Time:      1.000 seconds;       Data structure size:   0.1Mb
Mon Nov  9 15:29:29 2026
[Pilix@synopsys-unis alu_plus]$
```

Figura 6. *Simulación del sistema síncrono en VCS: 2,045 verificaciones con 0 errores.*

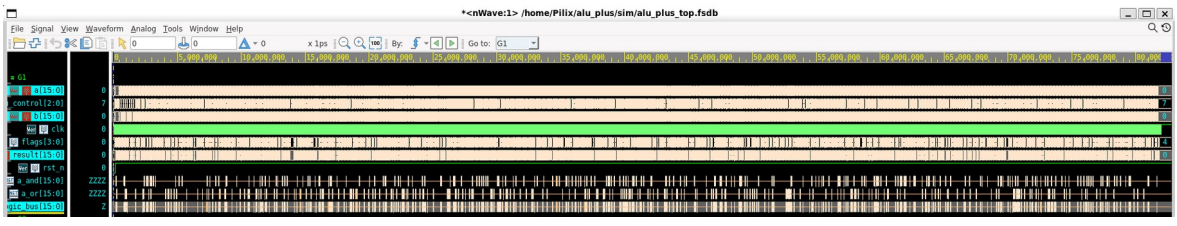


Figura 7. *Ondas de la simulación completa del sistema síncrono en Verdi (nWave).*

2.2 Tecnología y librerías

Elemento	Archivo usado

Las celdas M31 usan el sitio M31_CORE (filas de 3.36 µm) y los pines de alimentación VCCK y GNDK, que se conectaron a las redes VDD y VSS. La librería NDM para ICC2 se generó con `icc2_lm_shell` a partir del LEF, los archivos .db y el techfile del PDK.

2.3 Síntesis con Design Compiler

El script lee el RTL, aplica el SDC y ejecuta `compile_ultra -no_autoungroup`, que conserva la jerarquía para ver el área y el timing por bloque. El SDC define:

- Reloj de 40 ns (25 MHz) con incertidumbre de setup del 5 %.
- Input y output delay del 20 % del periodo.
- Transición de entrada de 0.3 ns, carga de salida de 0.05 pF y false path en el reset asíncrono.

Design Compiler 2025 exige declarar el diseño con `set_verification_top` antes de compilar; sin esa línea `compile_ultra` no se ejecuta y la netlist queda sin mapear.

2.4 Diseño físico con IC Compiler II

1. **Floorplan:** core cuadrado al 60 % de utilización y 66 filas de celdas; el bloque mide 243.2 × 241.8 µm.
2. **Alimentación:** anillo VDD/VSS alrededor del core, malla en METAL4 (vertical) y METAL5 (horizontal), y rieles en METAL1 sobre cada fila.
3. **place_opt:** coloca y legaliza 2,522 celdas y optimiza con reloj ideal.
4. **clock_opt:** construye el árbol de reloj y optimiza con el reloj propagado.
5. **route_auto:** routing global, asignación de tracks y routing detallado en 5 metales.
6. **route_opt:** optimiza con la resistencia y capacitancia reales de las rutas.
7. **Cierre:** inserción de fillers, verificaciones y salidas (netlist, SPEF, SDF y `alu_plus_top.gds` de 2.2 MB).

El kit de M31 no incluye el GDS interno de las celdas, por lo que el GDSII contiene el routing completo y el contorno (frame) de cada celda.

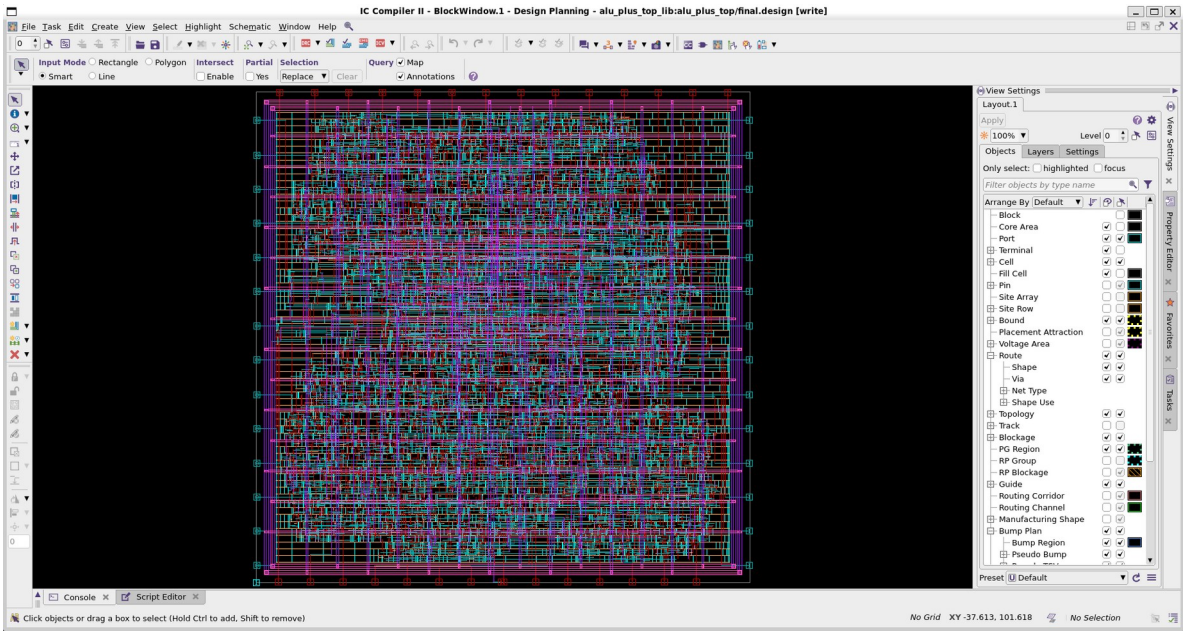


Figura 8. *Vista general del diseño final en la interfaz de IC Compiler II.*

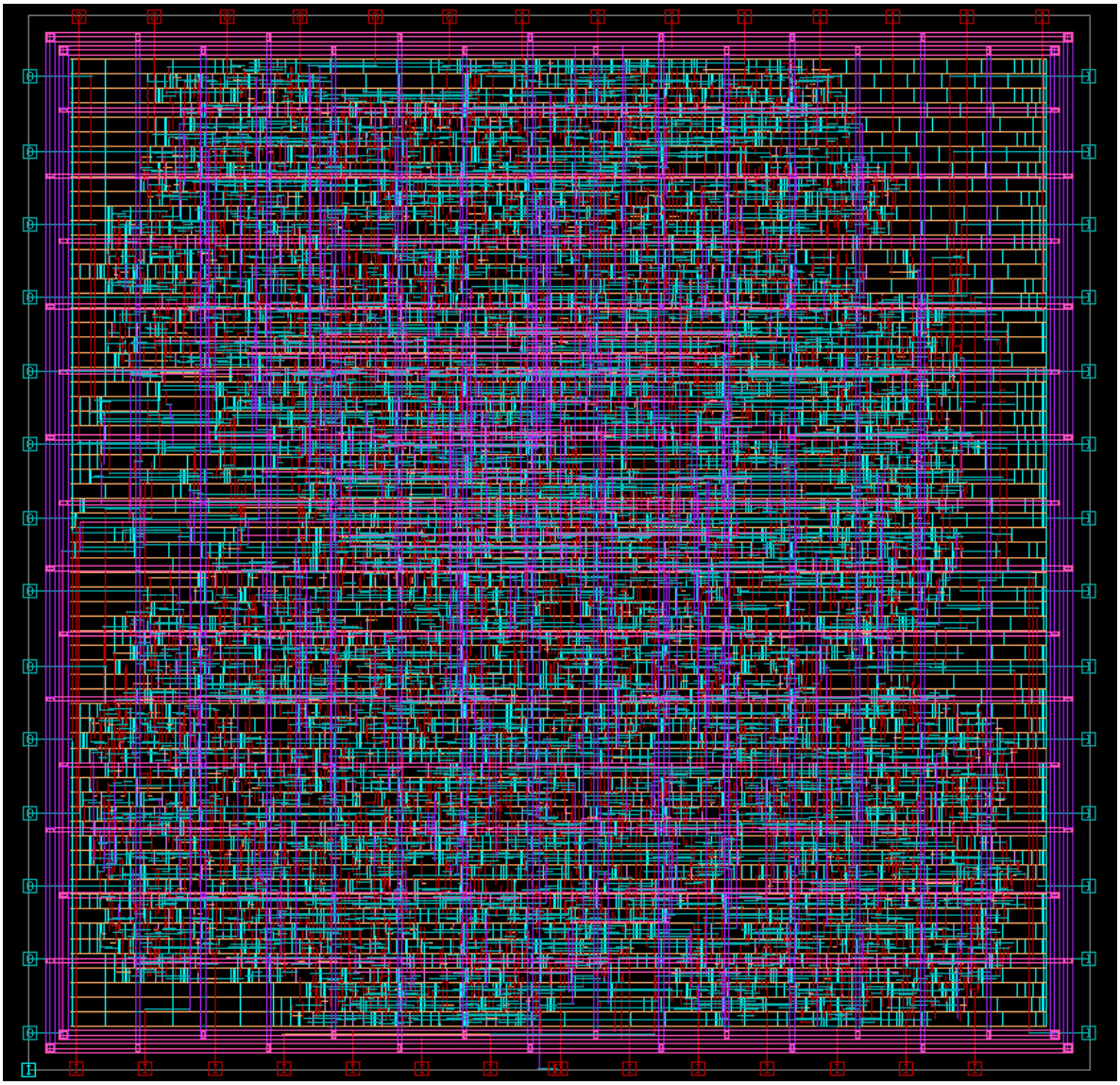


Figura 9. *Layout completo: anillo y malla de alimentación (rosa y morado) y routing de señales en los cinco metales.*

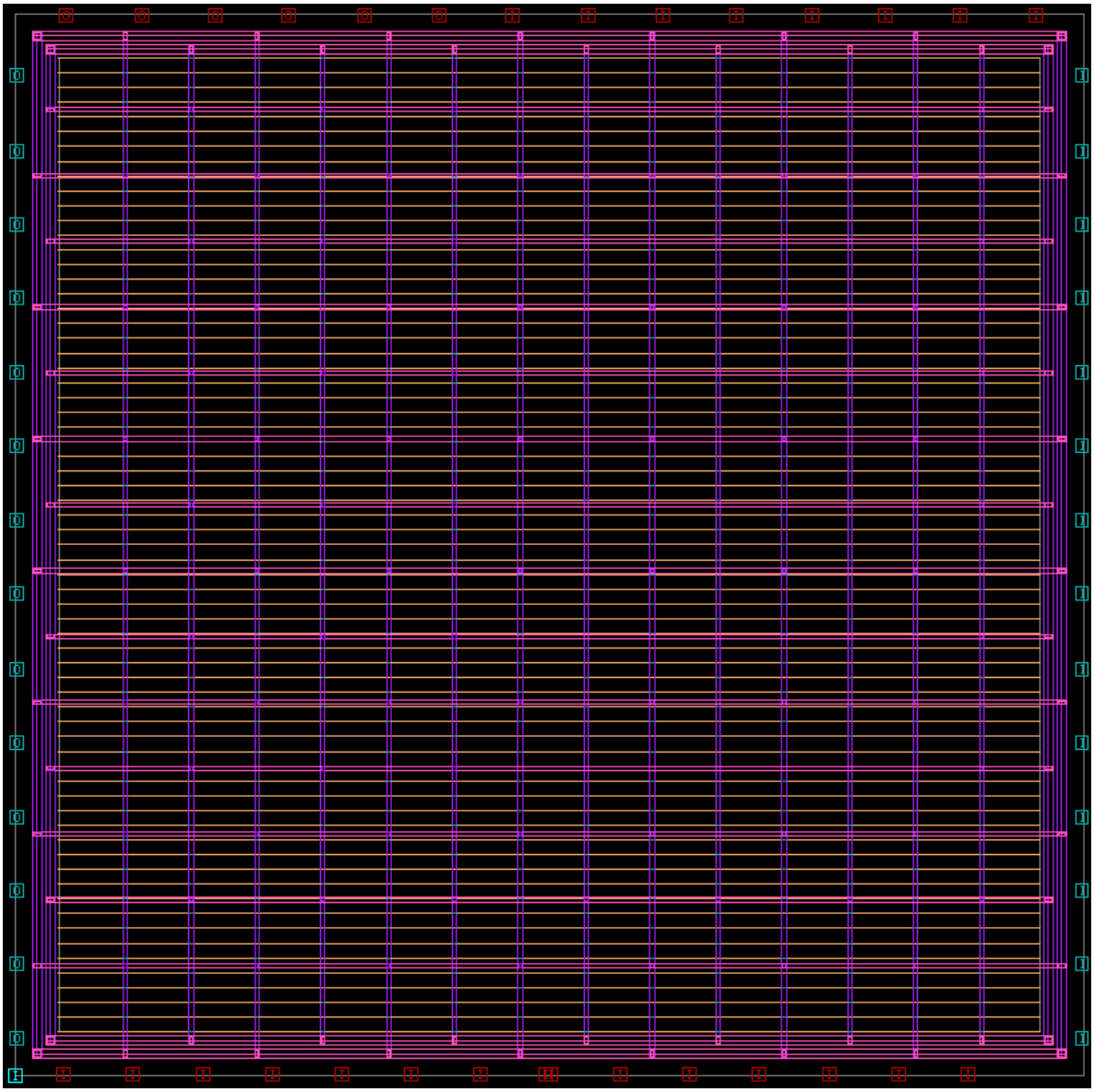


Figura 10. *Red de alimentación: malla en METAL4 (vertical) y METAL5 (horizontal), anillo alrededor del core y rieles de METAL1 en cada fila.*

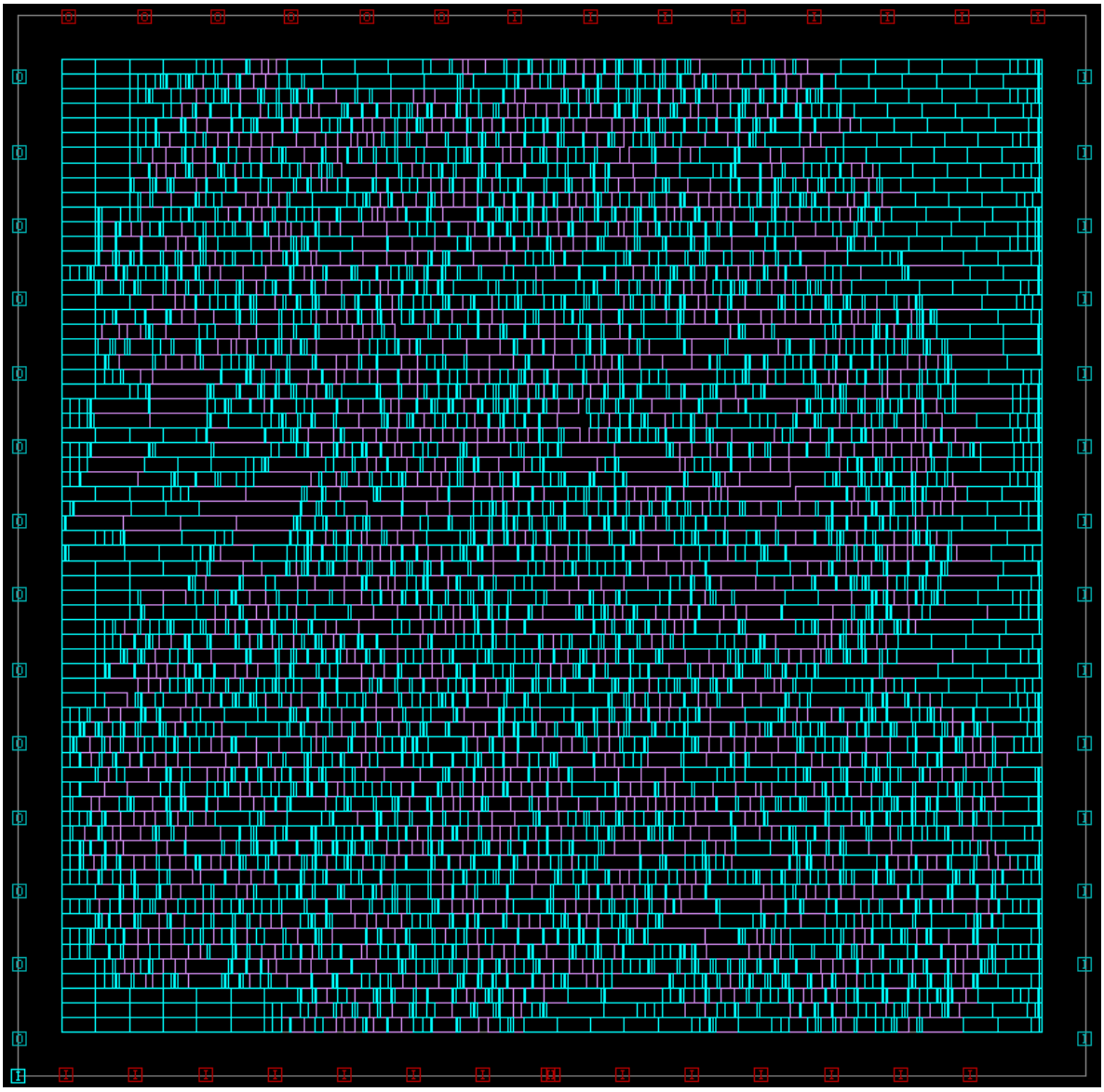


Figura 11. *Placement: celdas estándar acomodadas en las filas del core, sin mostrar el routing.*

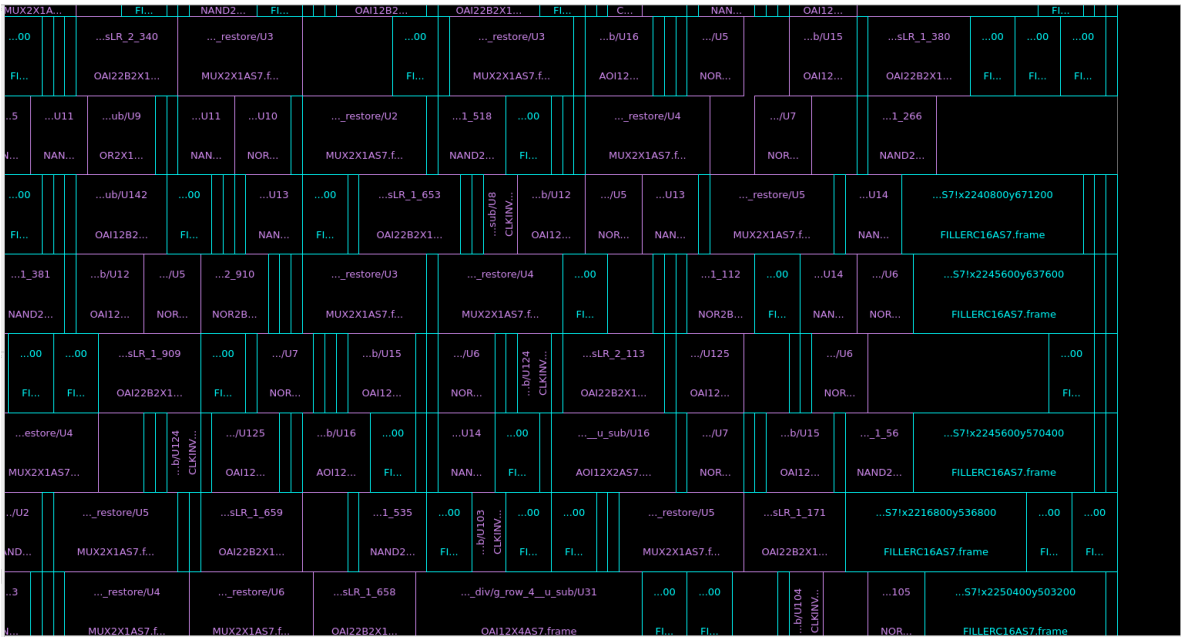


Figura 12. *Acercamiento a las celdas estándar M31 colocadas en las filas.*

Discusión

7.1 Síntesis (Design Compiler)

La síntesis produjo 2,902 celdas M31 en 29,988.4 µm² y cumple setup a 25 MHz con WNS = 0.00 ns y TNS = 0.00 ns. Quedan cinco violaciones pequeñas de hold en registros de entrada, que se corrigen en el diseño físico. Los reportes `report_qor`, `report_timing`, `report_area`, `report_power` y `report_constraint -all_violators` están en `syn/reports/`.

1. ¿Cuál es el área total del diseño y cuántas celdas contiene?

Métrica	Valor
Área total de celdas	29,988.4 µm²
Celdas totales	2,902
Celdas combinacionales	2,725
Celdas secuenciales	135 (incluyen los 55 flip-flops de los registros)
Macros / cajas negras	0

La mayor parte del área es el divisor, que equivale a 16 Prefix Adders de 17 bits más 16 multiplexores de restauración; el sumador principal y la lógica AND/OR son una fracción pequeña. La Figura 13 lo confirma en el layout: las celdas del divisor ocupan casi todo el chip.



Figura 13. *Celdas del divisor resaltadas en blanco: ocupan la mayor parte del área del chip.*

2. ¿Cuál es el periodo del reloj y qué frecuencia representa?

El periodo es 40 ns, que representa f = 1/T = 25 MHz. Como el WNS es 0.00 ns, el diseño usa justo el periodo disponible: Design Compiler recupera área en las rutas que tienen holgura hasta que el camino crítico queda exacto.

3. ¿Qué significan WNS y TNS? ¿El diseño cumple timing?

- **WNS** (Worst Negative Slack) es el slack de la peor ruta: tiempo requerido menos tiempo de llegada. Si es negativo, esa ruta llega tarde.

- **TNS** (Total Negative Slack) es la suma de los slacks negativos de todas las rutas que fallan; mide cuánto falla el diseño en total.

El diseño cumple setup: WNS = 0.00 ns y TNS = 0.00 ns. En hold, `report_constraint -all_violators` muestra cinco violaciones de −0.06 ns en `b_q_reg[11]` a `b_q_reg[15]`: el dato llega de la entrada al registro en 0.50 ns y se requieren 0.56 ns. Design Compiler no corrige hold por defecto; IC Compiler II lo hace en clock_opt, cuando conoce la latencia real del reloj.

4. ¿Cuál es el critical path? Startpoint, Endpoint y Path Group

Campo	Valor
Startpoint	b_q_reg[1] (flip-flop de entrada de B, flanco de subida de clk)
Endpoint	flags_reg[2] (bandera Z)
Path Group	clk
Slack	0.00 ns (MET)

5. ¿Qué operaciones o celdas provocan el mayor retardo en ese camino?

El camino sale de B y atraviesa el divisor de arreglo: 16 restas de 17 bits en serie, porque cada fila necesita el residuo de la anterior. Cada fila es una cadena de celdas NAND, NOR, AOI y XOR (generate, propagate y celdas prefijo) seguida de un `MUX2X1AS7` de restauración. Después pasa por el multiplexor 8:1 del resultado y termina en la NOR de 16 bits que calcula Z, por eso el endpoint es la bandera Z. Además, cada bit de B llega a las 16 filas, y ese fan-out alto hace más lentas sus transiciones.

6. ¿Qué optimizaciones implementarían?

Optimización	Por qué ayuda
Divisor secuencial o segmentado (pipeline)	Rompe las 16 restas en serie; la frecuencia sube varias veces a costa de latencia.
Ruta multiciclo para DIV/REM	Si se acepta esperar varios ciclos, el divisor deja de limitar la frecuencia de las demás operaciones.
Divisor non-restoring o SRT	Elimina el multiplexor de restauración de cada fila.
Prefix Kogge-Stone	Reduce el fan-out del Sklansky y el retardo por fila, con más área.
Calcular Z en paralelo	Evita que la NOR de 16 bits quede al final del camino más largo.
Buffering de B	Reduce el fan-out de la señal que alimenta las 16 filas.

Optimización	Por qué ayuda
set_fix_hold en síntesis	Corrige desde Design Compiler las violaciones de hold de los registros de entrada.

7.2 Implementación física (IC Compiler II)

El layout final pasó routing (0 DRC), legalidad, conectividad de alimentación y LVS sin errores. Tras una pasada extra de `route_opt` el diseño cumple todo el timing a 25 MHz: setup con WNS = 0.00 ns y 0 endpoints fallando, hold con +0.89 ns, y 0 violaciones de transición y capacitancia.

1. Etapas ejecutadas y reportes

Se ejecutó `place_opt` → `clock_opt` → `route_auto` → `route_opt` y, después de cada etapa, `report_qor`, `report_timing` (setup y hold), `report_constraints -all_violators` y `report_utilization`, guardados en `pnr/reports/`.

2. ¿Cómo cambia el WNS entre las diferentes etapas?

Etapa	WNS setup (ns)	TNS (ns)	Endpoints que fallan	Área de celdas (μm²)
Síntesis (DC)	0.00	0.00	0	29,988.4
place_opt	+0.06	0.00	0	29,391.7
clock_opt	+0.01	0.00	0	28,806.2
route_auto	−0.56	−0.60	2	28,806.2
route_opt	−0.01	−0.01	1	28,885.3
Final con fillers (antes de corregir)	−0.06	−0.06	1	28,885.3
route_opt incremental (corrección)	0.00	0.00	0	28,909.4

- **place_opt (+0.06 ns):** ya con distancias reales entre celdas, el optimizador cumple con un margen pequeño.
- **clock_opt (+0.01 ns):** el reloj pasa de ideal a propagado; aparecen latencia y skew y el margen se reduce.
- **route_auto (−0.56 ns):** es el peor punto; las rutas reales tienen desvíos y más capacitancia que la estimada, y dos endpoints fallan.

- **route_opt (−0.01 ns):** con la RC real de las rutas, redimensiona celdas y recupera casi todo el timing.
- **Final con fillers (−0.06 ns):** al insertar fillers y re-extraer parásitos quedó un endpoint 0.06 ns tarde.
- **route_opt incremental (0.00 ns):** una pasada más de optimización post-route sobre el diseño final redimensionó las celdas de esa ruta (+24 µm² de área) y eliminó la violación, sin introducir DRC ni errores de LVS.

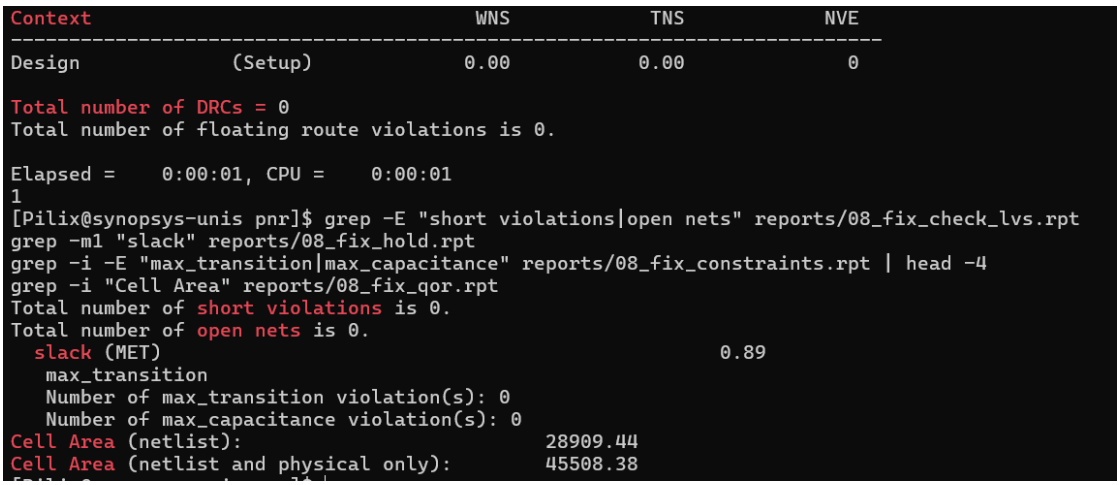


Figura 14. *Resultado tras el route_opt incremental: WNS = 0.00 ns sin endpoints fallando, 0 DRC, LVS sin cortos ni redes abiertas, hold +0.89 ns y 0 violaciones de transición y capacitancia.*

3. ¿Cómo afectan el clock skew y las latencias del reloj al timing?

Métrica del árbol de reloj	Valor
Flip-flops conectados (sinks)	55
Niveles del árbol	2
Repetidores insertados	1
Latencia máxima	0.40 ns
Skew global	0.02 ns
Longitud del cable de reloj	931.3 µm

La latencia es el tiempo que tarda el reloj en llegar desde su fuente al pin de cada flip-flop; si es igual en todos, se cancela. El skew es la diferencia de latencia entre el flip-flop que lanza el dato y el que lo captura:

$$slack(setup) = T + (latencia\ captura - latencia\ lanzamiento) - t(datos) - t(setup)$$

Un skew positivo ayuda a setup pero perjudica a hold. Aquí el árbol quedó casi balanceado: 0.02 ns de skew es el 0.05 % del periodo, así que prácticamente no afecta el timing. La latencia de 0.40 ns sí importa en las rutas que vienen de los puertos de entrada: el reloj llega 0.40 ns tarde a los registros que capturan A y B, lo que empeora el hold de esas rutas cortas, por eso clock_opt inserta retardo en ellas. El placement colocó los registros de entrada cerca de los pines de entrada y los de salida cerca de los pines de salida, lo que permitió un árbol con un solo repetidor.

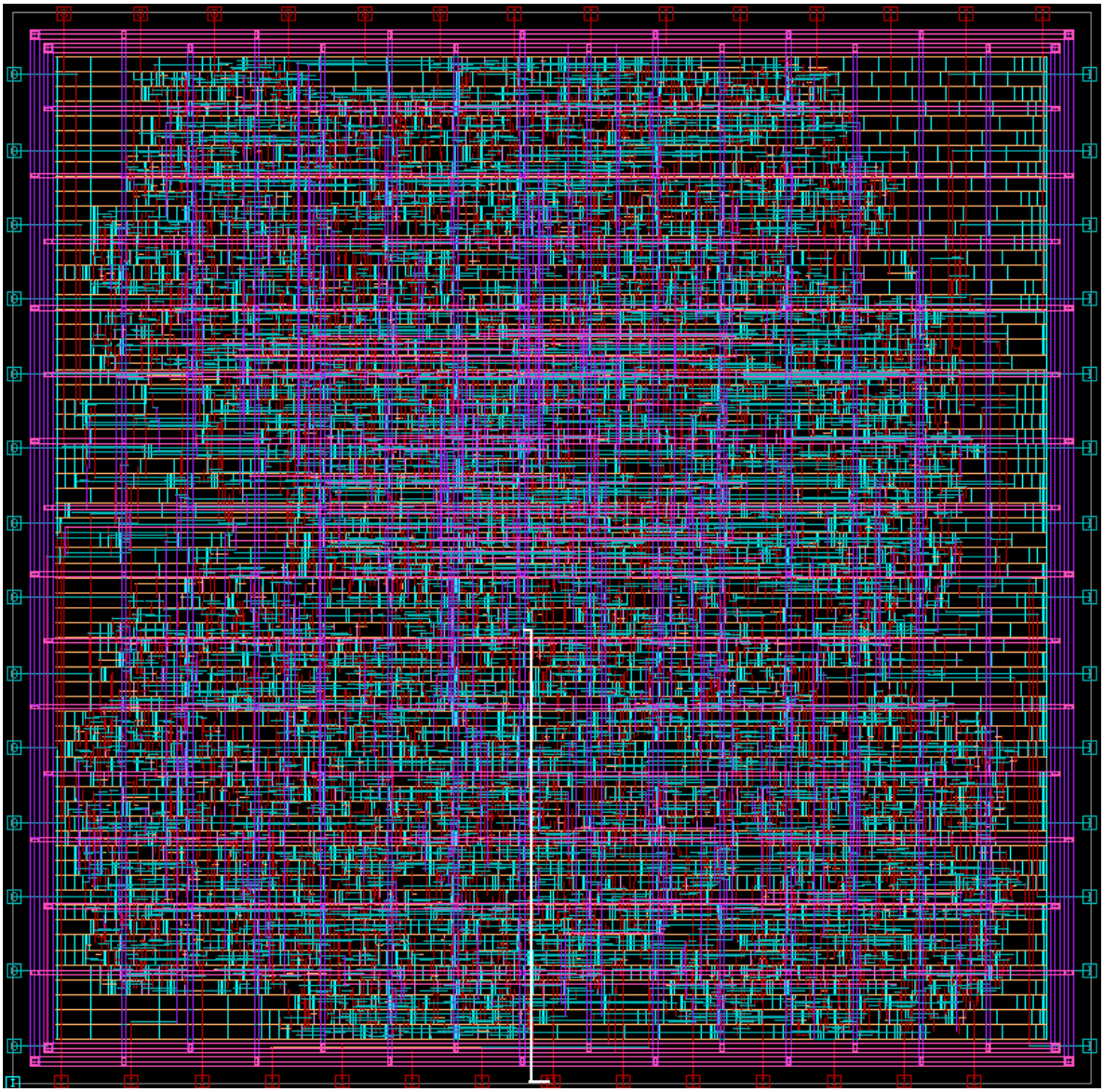


Figura 15. *Red principal del reloj: entra por el pin clk y llega al buffer insertado por el CTS.*

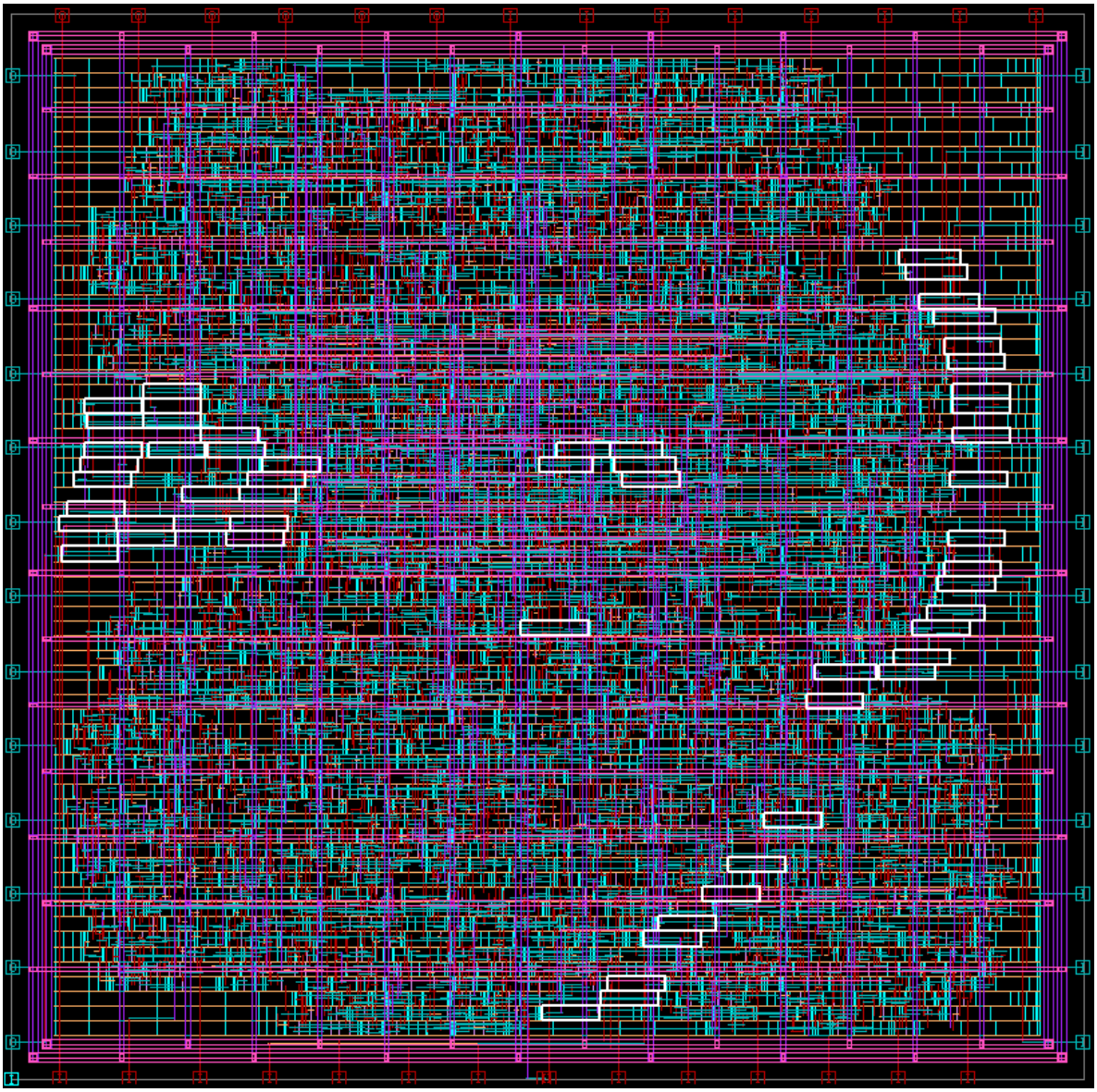


Figura 16. *Los 55 flip-flops alimentados por el árbol de reloj: registros de entrada a la izquierda y de salida a la derecha, cerca de sus pines.*

4. ¿Aumentó el área respecto a Design Compiler? ¿Por qué?

El área de lógica bajó 3.6 %, de 29,988.4 µm² en Design Compiler a 28,909.4 µm² al final: IC Compiler II re-optimizó con distancias y cargas reales y redujo el tamaño de celdas que no estaban en el camino crítico. El área total de celdas sí subió a 45,508.4 µm² al incluir 3,800 celdas filler (4,227 insertadas menos 427 retiradas por cortos o espaciado), que rellenan los huecos de las filas para dar continuidad a los pozos y a los rieles de alimentación.

Área	**Valor**
Bloque (die)	243.2 × 241.8 µm

Área	Valor
Core	49,496.8 µm²
Celdas lógicas	28,909.4 µm² (58 % del core)
Celdas lógicas + fillers	45,508.4 µm²

5. ¿Existen violaciones de setup, hold, capacitancia o transición?

Tipo	Resultado final
Setup	Sin violaciones: el endpoint de −0.06 ns se corrigió con un route_opt incremental; WNS = 0.00 ns y TNS = 0.00 ns.
Hold	Sin violaciones: las cinco de la síntesis quedaron corregidas y el peor slack de hold es +0.89 ns.
Transición (max_transition)	0 violaciones
Capacitancia (max_capacitance)	0 violaciones

6. ¿Qué muestran las verificaciones de routing, legalidad y conexiones de alimentación?

Verificación	Comando	Resultado
Routing	check_routes	0 DRC en todos los metales
Legalidad	check_legality	Aprobada: celdas en sitios válidos y sin traslapes
Alimentación	check_pg_connectivity	0 cables flotantes y 0 celdas sin VDD/VSS
LVS	check_lvs	0 cortos, 0 redes abiertas y 0 rutas flotantes
Antenas	check_routes	No evaluadas: las reglas de antena del PDK no se cargaron

El layout es consistente con la netlist y cumple las reglas de routing.

7. Potencia

La potencia total después del routing es 1.12 mW: 0.63 mW de potencia interna, 0.50 mW de conmutación y 5.15 nW de fuga. En 150 nm la fuga es despreciable y domina la potencia dinámica; el divisor conmuta aunque la operación sea otra, por eso aislar sus operandos ahorraría potencia.

8. ¿Cómo se transformó el RTL en celdas, incluido el bus triestado?

RTL	Celdas M31
always_ff con reset asíncrono	DFFRBQX1AS7 / DFFRBQX2AS7
Multiplexores con operador ternario	MUX2X1AS7 y compuertas AND/NAND
Prefix Adder	NAND, NOR, AOI, XOR y sumadores completos FAX1AS7
Buffers triestado (? : 'z)	TRIINVX1AS7, inversores triestado reales
Árbol de reloj	CLKBUFX8AS7
Relleno de filas	FILLER*AS7 y FILLERC*AS7

Los `? : 'z` se convirtieron en celdas triestado físicas, así que el aislamiento de AND/OR del inciso 6 existe en el chip: cuando la operación no es AND ni OR, esas celdas dejan sus salidas en alta impedancia. El placement agrupó estas celdas en una sola región, cerca de la unidad lógica y del multiplexor de resultado, para acortar sus conexiones.

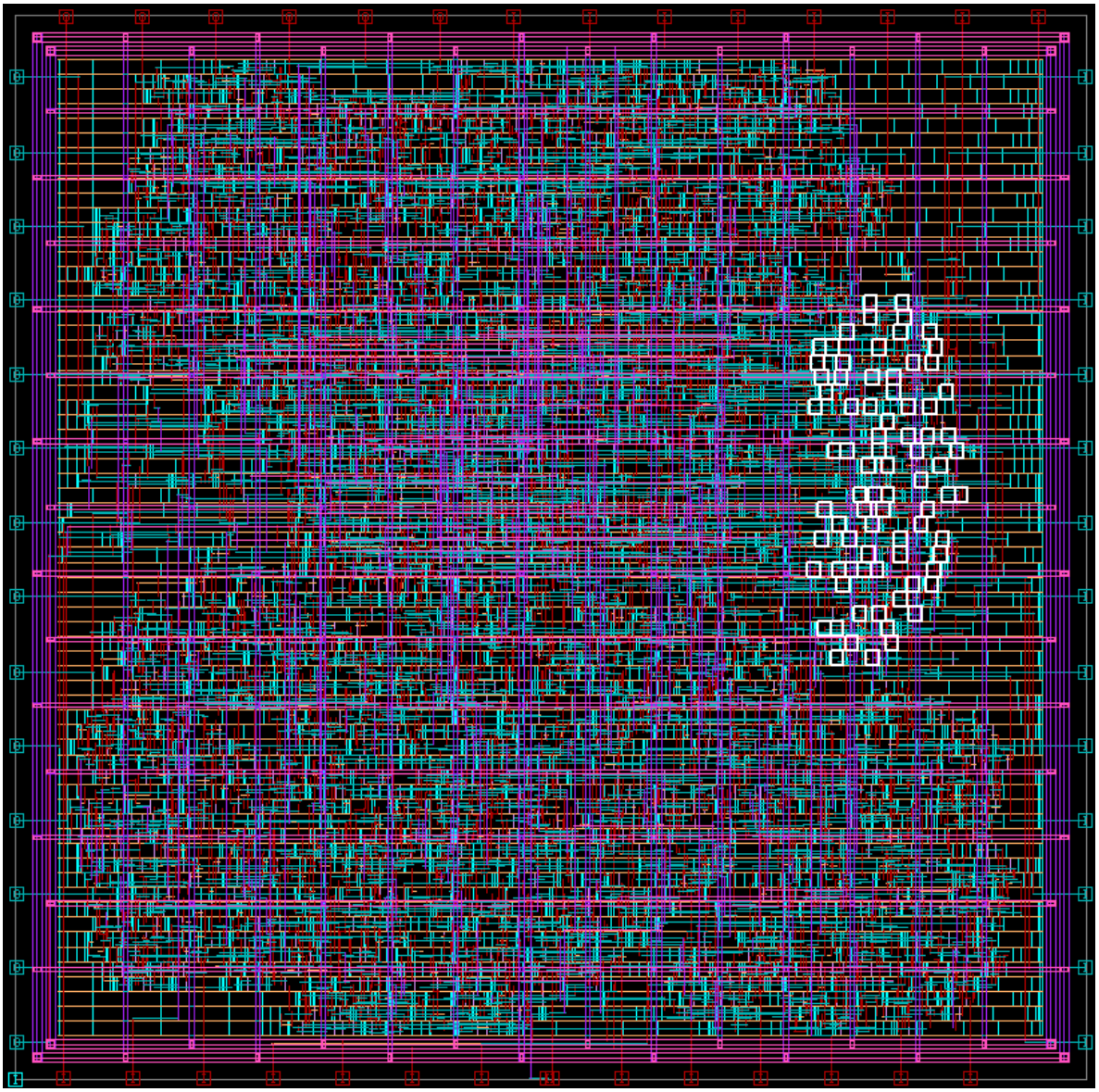


Figura 17. *Celdas triestado TRIINVX1AS7 resaltadas: agrupadas cerca de la unidad lógica AND/OR.*

Conclusiones

- Una sola descripción parametrizable de la ALU+ funcionó sin cambios en 8, 16 y 32 bits, verificada con 0 errores (exhaustiva a 8 bits), y el sistema síncrono de 16 bits pasó 2,045 vectores en VCS.
- El Prefix Adder reduce la propagación del carry a $\log_2 N$ niveles, pero el divisor de arreglo encadena 16 restas y domina el área y el camino crítico, que va de `b_q_reg[1]` a la bandera Z; es el primer bloque a optimizar.
- Describir los multiplexores con el operador ternario y el bus con `? : 'z` permitió que Design Compiler los mapeara a celdas MUX2 y a inversores triestado TRIINVX1AS7, de modo que el aislamiento de AND/OR existe físicamente en el chip.
- A 25 MHz el diseño cumple setup tras la síntesis. En el diseño físico el WNS pasó de +0.06 ns en placement a −0.56 ns en el routing inicial, y la optimización post-route lo llevó a 0.00 ns, cumpliendo a 25 MHz: las rutas reales cambian el timing y la optimización post-route es indispensable.
- El árbol de reloj quedó casi balanceado (skew de 0.02 ns y latencia de 0.40 ns) y el área de lógica bajó 3.6 % respecto a la síntesis; el área total subió por las celdas filler.
- El layout final en GF 150 nm con celdas M31 pasó routing (0 DRC), legalidad, conectividad de alimentación y LVS, y se exportó a GDSII.